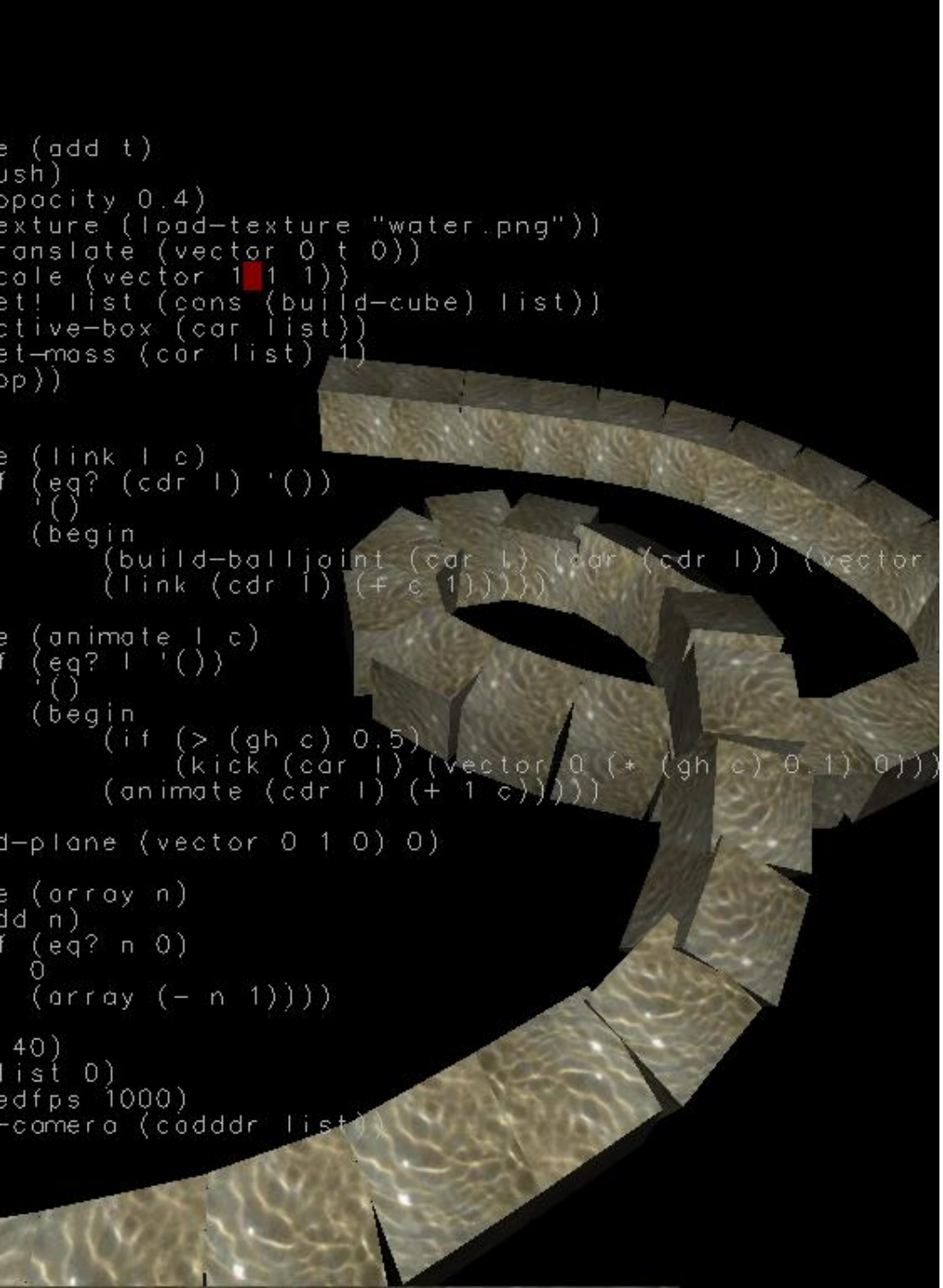


算法分析与设计
Analysis and Design of Computer Algorithms
动态规划

李纲
宁波大学
信息科学与工程学院



学习要点:

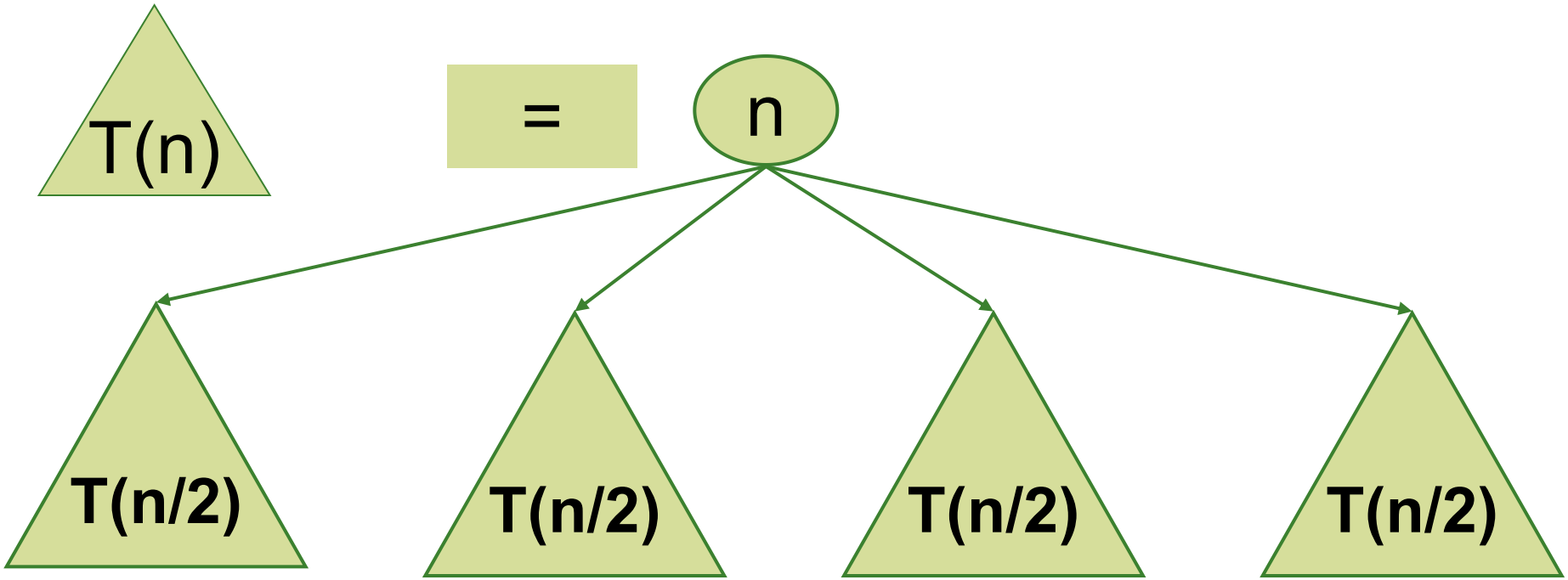
- 理解动态规划算法的概念。
- 掌握动态规划算法的基本要素
- （1）最优子结构性质
- （2）重叠子问题性质
- 掌握设计动态规划算法的步骤。
- (1)找出最优解的性质，并刻划其结构特征。
- (2)递归地定义最优值。（上->下）
- (3)以自底向上的方式计算出最优值。（下->上）
- (4)根据计算最优值时得到的信息，构造最优解。



- 通过应用范例学习动态规划算法设计策略。
- （1）矩阵连乘问题；
- （2）最长公共子序列；
- （3）最大子段和
- （4）凸多边形最优三角剖分；
- （5）多边形游戏；
- （6）图像压缩；
- （7）电路布线；
- （8）流水作业调度；
- （9）背包问题；
- （10）最优二叉搜索树。

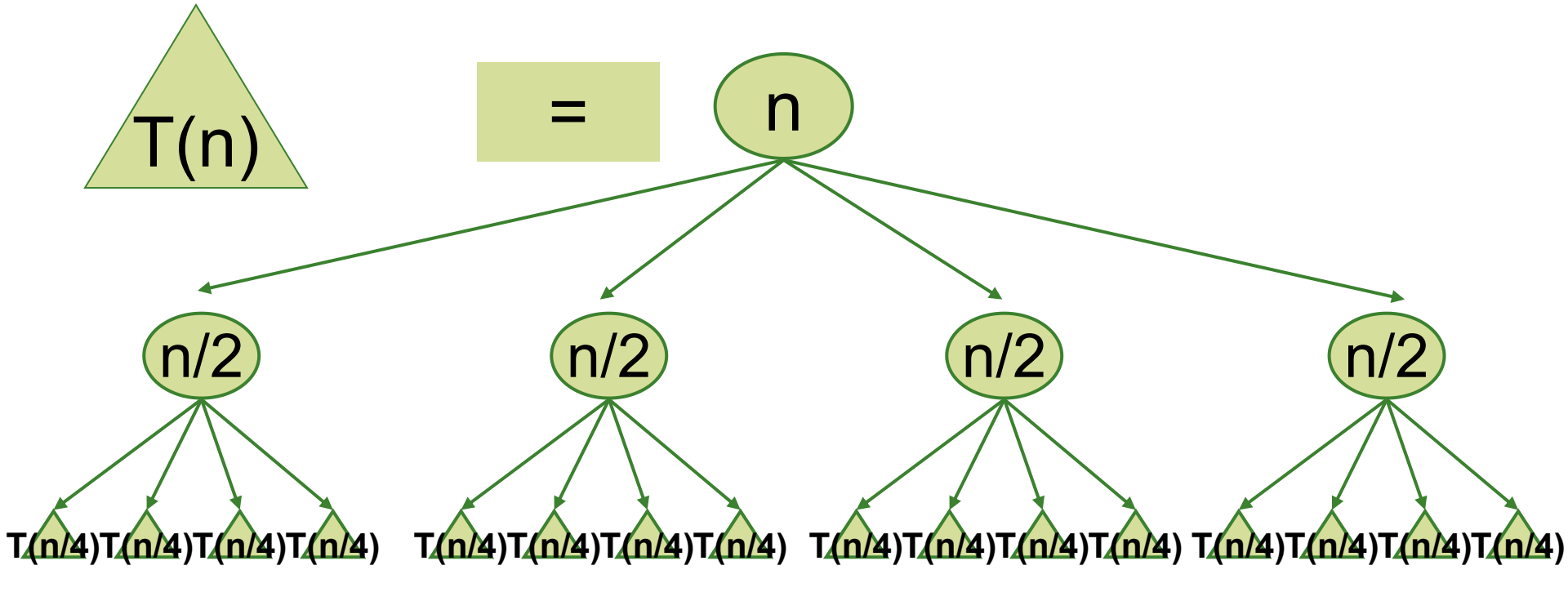
算法总体思想

- 动态规划算法与分治法类似，**其基本思想也是将待求解问题分解成若干个子问题**



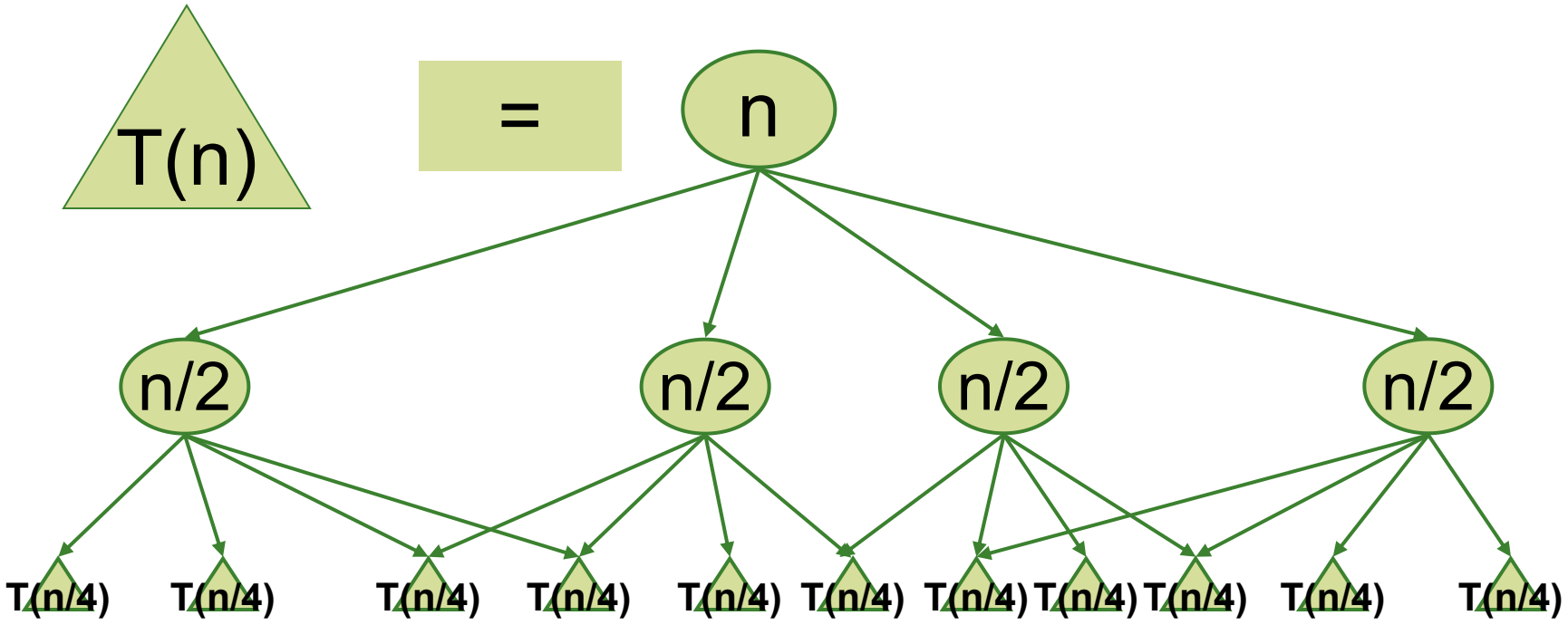
算法总体思想

- 但是经分解得到的**子问题往往不是互相独立的**。不同子问题的数目常常只有多项式量级。在用分治法求解时，**有些子问题被重复计算了许多次**。



算法总体思想

- 如果能够**保存已解决的子问题的答案**，而在需要时再找出已求得的答案，就可以**避免大量重复计算**，从而得到多项式时间算法。





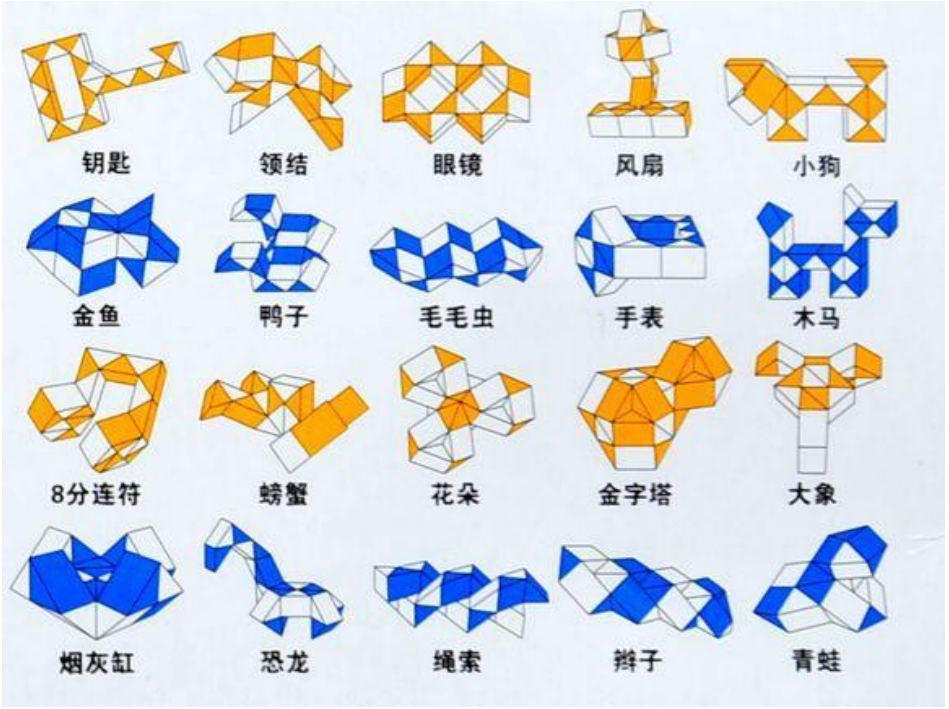
动态规划基本步骤

- 找出最优解的性质，并刻划其结构特征。
- 递归地定义最优值。
- 以**自底向上的方式计算出最优值**。
- 根据计算最优值时得到的信息，构造最优解。

动态规划基本步骤

- 魔尺（每一个节点都有可能转折，不同的转折决定了后面的形状）





完全加括号的矩阵连乘积



- 完全加括号的矩阵连乘积可递归地定义为：
  (1) 单个矩阵是完全加括号的；
  (2) 矩阵连乘积A是**完全加括号**的，则A可表示为2个完全加括号的矩阵连乘积B和C的乘积并加括号，即A=(BC)
- 设有四个矩阵A、B、C、D ，
- 它们的维数分别是：

$$A = 50 \times 10 \quad B = 10 \times 40 \quad C = 40 \times 30 \quad D = 30 \times 5$$

- 总共有五中完全加括号的方式

$$(A((BC)D)) \quad (A(B(CD))) \quad ((AB)(CD)) \quad (((AB)C)D) \quad ((A(BC))D)$$

16000, 10500, 36000, 87500, 34500

=(10*40*30)+10*30*5+50*10*5

BC *D A*



矩阵连乘问题

- 给定n个矩阵$\{A_1, A_2, \ldots, A_n\}$其中 与 A_i 是 A_{i+1}可乘的， $i = 1,2,\ldots,n-1$ 。考察这n个矩阵的连乘积

$$A_1 A_2 \ldots A_n$$

- 由于矩阵乘法满足结合律，所以计算矩阵的连乘可以**有许多不同的计算次序**。这种计算次序可以用加括号的方式来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积。



矩阵连乘问题

给定n个矩阵 $\{A_1,A_2,\ldots,A_n\}$，其中A_i与A_{i+1}是可乘的，i=1，2...，n-1。如何确定计算矩阵连乘积的计算次序，使得依此次序计算矩阵连乘积需要的数乘次数最少。

◆**穷举法**：列举出所有可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

算法复杂度分析：

对于n个矩阵的连乘积，**设其不同的计算次序为P(n)。**

由于每种加括号方式都可以分解为两个子矩阵的加括号问题：$(A_1\ldots A_k)(A_{k+1}\ldots A_n)$可以得到关于P(n)**(catalan数)**的递推式如下：

$$P(n)=\begin{cases}1 & n=1\\ \sum_{k=1}^{n-1}P(k)P(n-k) & n>1\end{cases}\Rightarrow P(n)=\Omega(4^n/n^{3/2})$$



矩阵连乘问题

令h(0)=1,h(1)=1，catalan数满足递归式：
h(n)= h(0)*h(n-1) + h(1)*h(n-2) + ... + h(n-1)h(0) (其中n>=2)
这是n阶递推关系

◆1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790（亿）, 477638700, 1767263190, 6564120420, 24466267020, 91482563640, 343059613650, 1289904147324, 4861946401452, ...

怎么偷懒?
无排序的数组，求最大数？ max={max与当前比较} O(n)
二分得到最大的数 O(logn)

斐波那契数列：a[n]=a[n-1]+a[n-2] 只跟前2个有关

比如足球比赛，各洲出线（预选赛）-小组赛-1/8-1/4-1/2-决赛；只跟前一步的几个状态有关
如果大家都发挥正常，那么：冠军是否真的冠军？亚军或季军是否真的亚军或季军?



矩阵连乘问题

计算量对比：

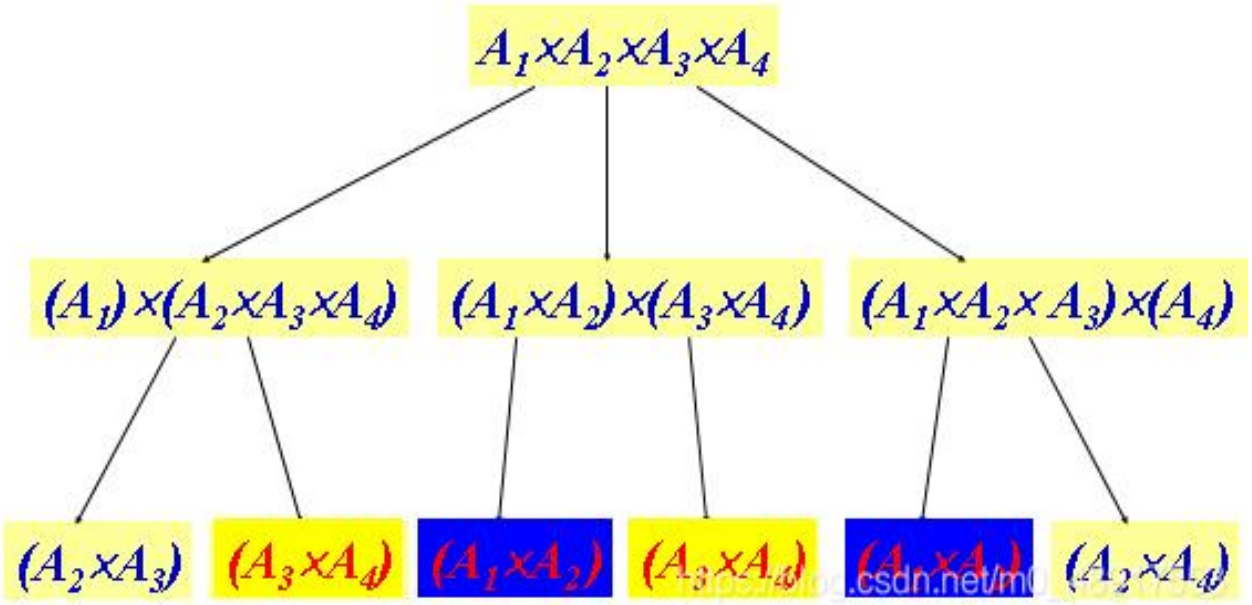
全局<---PK---->仅考虑前一步与当前的递推关系

比如下棋：
（1）可以悔棋-------->可以悔棋任意步
（2）不可以悔棋/不需要悔棋

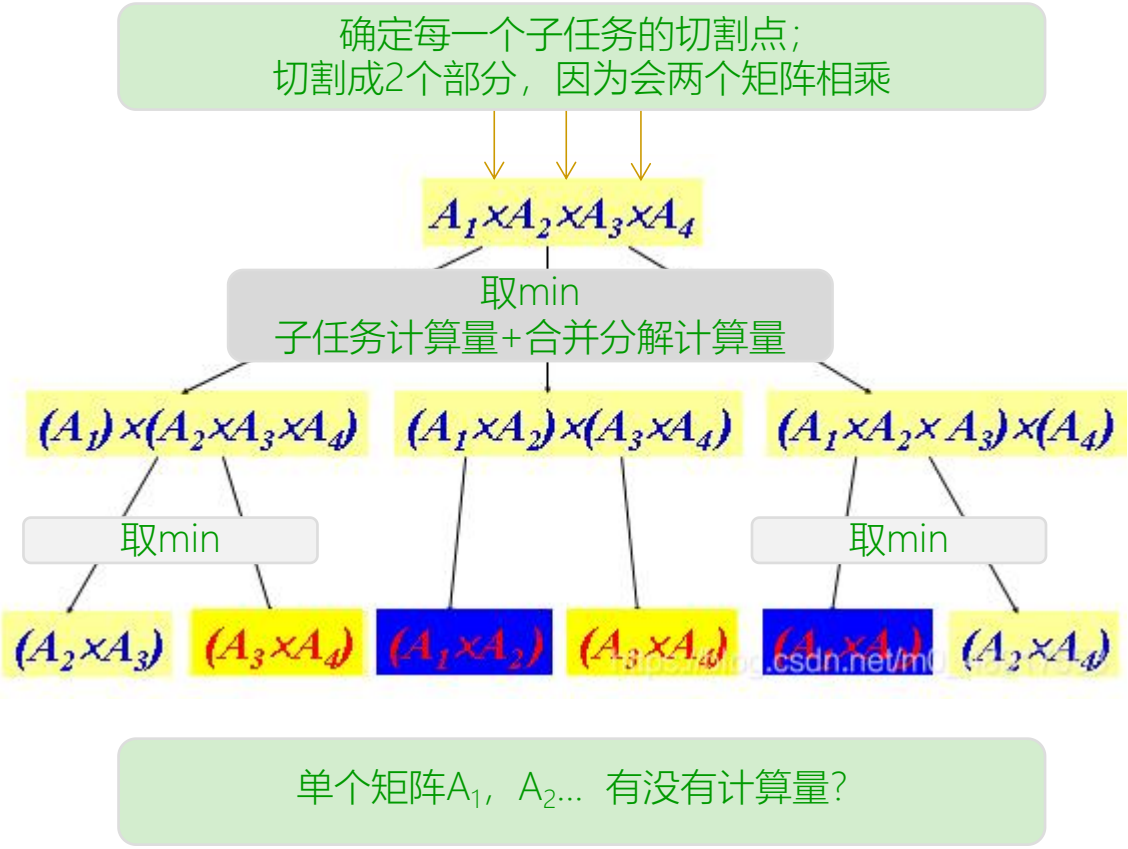
比如迷宫，最短路径：
（1）所有组合可能
（2）但是对于某个点，可以只考虑能够到达这一步有几个可能性（递推）；
前一步的来源直接影响这一步；
至于其他的，没直接关系

矩阵连乘问题

只考虑上下两步之间的关系，不要复杂化
原则：你每次只能算两个矩阵的乘积
然后：有几种状态->子问题



矩阵连乘问题


确定每一个子任务的切割点；
切割成2个部分，因为会两个矩阵相乘
$A_1 \times A_2 \times A_3 \times A_4$
取min
子任务计算量+合并分解计算量
$(A_1) \times (A_2 \times A_3 \times A_4)$
$(A_1 \times A_2) \times (A_3 \times A_4)$
$(A_1 \times A_2 \times A_3) \times (A_4)$
取min
取min
$(A_2 \times A_3)$
$(A_3 \times A_4)$
$(A_1 \times A_2)$
$(A_3 \times A_4)$
$(A_2 \times A_4)$
单个矩阵A_1，A_2... 有没有计算量?

矩阵连乘问题



◆动态规划

将矩阵连乘积 $A_iA_{i+1}\ldots A_j$ 简记为A[i:j]，这里i≤j

考察计算A[i:j]的最优计算次序。设这个计算次序在矩阵A_k和A_{k+1}之间将矩阵链断开，i≤k<j，则其相应完全加括号方式为 $(A_iA_{i+1}\ldots A_k)(A_{k+1}A_{k+2}\ldots A_j)$

计算量（只跟前一步的几个状态有关）：

(1) A[i:k]的计算量加上A[k+1:j]的计算量

(2) 再加上A[i:k]和A[k+1:j]相乘的计算量

每一步，实际上就是2个矩阵的相乘；也只能是2个矩阵相乘（2个子问题）



分析最优解的结构

- **特征：计算A[i:j]的最优次序所包含的计算矩阵子链 A[i:k]和A[k+1:j]的次序也是最优的。**

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。

- 每个阶段有一个状态/或多个，但不需要比较最优->递推;
- 每个阶段的最优状态都是由上一个阶段的最优状态+本次最优->贪心;
- 每个阶段的最优状态是由之前所有阶段的状态的组合得到的->搜索;
- 每个阶段的最优状态可以从之前某个阶段的某个或某些状态直接得到而不管之前这个状态(最x值)是如何得到的->动态规划。



建立递归关系

- 设计算A[i:j]，1≤i≤j≤n，所需要的最少数乘次数m[i,j]，则原问题的最优值为m[1,n]　　（P_i表示矩阵的维度,只有一维）
- 当i=j时，A[i:j]=A_i，因此，m[i,i]=0，i=1,2,...,n
- 当i<j时，

$(A_iA_{i+1}\cdots A_k)(A_{k+1}A_{k+2}\cdots A_j)$

$A=50\times10$　$B=10\times40$　$C=40\times30$　$D=30\times5$

$$m[i,j]=m[i,k]+m[k+1,j]+p_{i-1}p_kp_j$$

这里　A_i的维数为　$p_{i-1}\times p_i$

- 可以递归地定义m[i,j]为：

(1) A[i:k]的计算量加上A[k+1:j]的计算量
(2) 再加上A[i:k]和A[k+1:j]相乘的计算量

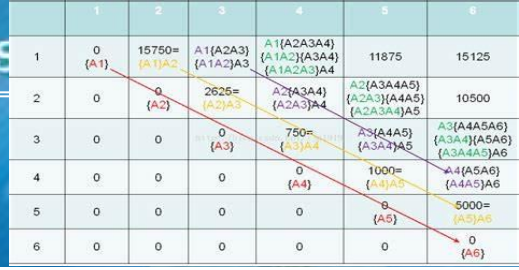
$$m[i,j]=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{m[i,k]+m[k+1,j]+p_{i-1}p_kp_j\} & i<j\end{cases}$$

$A=50\times10$　$B=10\times40$　$C=40\times30$　$D=30\times5$

当K位于哪个位置时值最佳；
见下页例子

k 的位置只有 $j-i$ 种可能

用动态规划法求最优解

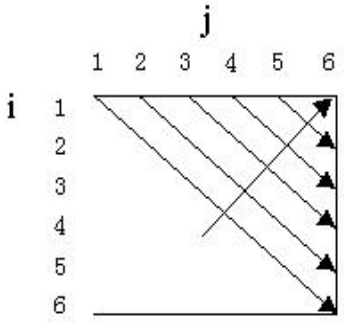


r=4

$$m[2][5]=\min\begin{cases} m[2][2]+m[3][5]+p_1p_2p_5 = 0+2500+35\times15\times20 = 13000 \\ m[2][3]+m[4][5]+p_1p_3p_5 = 2625+1000+35\times5\times20 = 7125 \\ m[2][4]+m[5][5]+p_1p_4p_5 = 4375+0+35\times10\times20 = 11375 \end{cases}$$

算法复杂度分析：

算法**matrixChain**的主要计算量取决于算法中对**r，i和k的3重循环。**循环体内的计算量为O(1)，而3重循环的总次数为$O(n^3)$。因此算法的计算时间上界为$O(n^3)$。算法所占用的空间显然为$O(n^2)$。



(a) 计算次序

j

i	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) m[i][j]

j

i	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

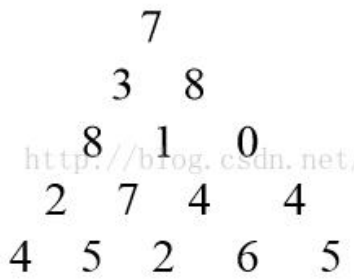
(c) s[i][j]

无后效性+最优子结构

- 上述的状态转移方程中，等式右边不会用到下标超出左边i或者右边j的值，这是"无后效性"的通俗上的数学定义；
- 当下：无关过去，无关未来

- 符合这种定义的状态定义，我们可以说它具有“最优子结构”的性质，在动态规划中我们要做的，就是找到这种“最优子结构”。

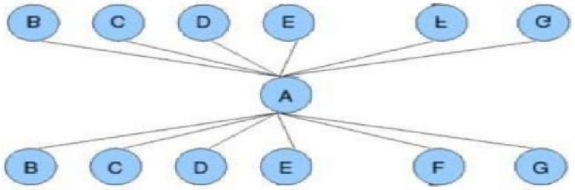
- 下例比较自顶向下的贪心和自底向上的动态规划



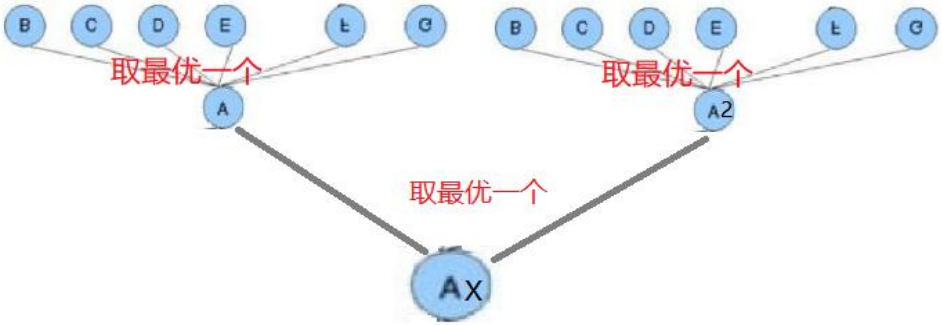
http://blog.csdn.net/

连续型子问题和离散型子问题

- 搜索：A上一层有m个节点，A下一层有n个节点，则通过A的可能路径有m*n



- 动态规划：A的上一层相关节点，会取一个最优点，构成A的最优值；然后A的下一层亦然。



连续型子问题和离散型子问题



- 动态规划从下一层的最优得出本节点的最优
- 我们简单的把下一层的子问题分为：伪连续型和离散型
- 比如矩阵，连续的乘法串从哪个位置断开划分子问题
- 而上例的求和，或者01背包，我们可以称为离散型子问题



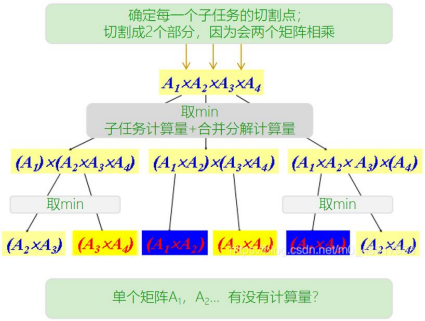
计算最优值

- 对于1≤i≤j≤n不同的有序对(i,j)对应于不同的子问题。因此，不同子问题的个数最多只有

$$\binom{n}{2} + n = \Theta(n^2)$$

- 由此可见，**在递归计算时，许多子问题被重复计算多次**。这也是该问题可用动态规划算法求解的**又一显著特征**。
- 用动态规划算法解此问题，可依据其递归式以自底向上的方式进行计算。在计算过程中，保存已解决的子问题答案。每个子问题只计算一次，而在后面需要时只要简单查一下，从而避免大量的重复计算，最终得到多项式时间的算法

用动态规划法求最优解




```
void MatrixChain(int *p, int n, int **m, int **s){
    for (int i = 1; i <= n; i++) m[i][i] = 0;
    for (int r = 2; r <= n; r++)              //r为i,j的间隔(矩阵个数从2到n个变化)
      for (int i = 1; i <= n - r+1; i++) {    //保持间隔为r，遍历一次
        int j=i+r-1;          //(对上层循环的矩阵,变化分割位置，求[i][j]的最优计算)
        m[i][j] = m[i+1][j]+ p[i-1]*p[i]*p[j];  //间隔为1，计算次数放到m[][]
        s[i][j] = i;                            //s[][],分割位置在i，即k=i
        for (int k = i+1; k < j; k++) { //间隔逐渐增大，找出m[i][j]的最优分割
          int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
          if (t < m[i][j]) { m[i][j] = t; s[i][j] = k;}
        }
      }
}
```

m[i][i]+

自底向上求出任意间隔的计算代价

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25



矩阵连乘问题

找出一下的递推关系：

1.括号化问题。
矩阵连乘： P=a0×a1×a2×a3×……×an，共有（n+1）项，依据乘法结合律，不改变其顺序，只用括号表示成对的乘积，试问有几种括号化的方案？(h(n)种)
类似题目：有N个节点的二叉树共有多少种情形？

2.出栈次序问题。
一个栈(无穷大)的进栈序列为1,2,3,..n,有多少个不同的出栈序列?
类似题目：有2n个人排成一行进入剧场。入场费5元。其中只有n个人有一张5元钞票，另外n人只有10元钞票，剧院无其它钞票，问有多少中方法使得只要有10元的人买票，售票处就有5元的钞票找零？(将持5元者到达视作将5元入栈，持10元者到达视作使栈中某5元出栈)

3.将多边行划分为三角形问题。
将一个凸多边形区域分成三角形区域的方法数?
类似题目：一位大城市的律师在她住所以北n个街区和以东n个街区处工作。每天她走2n个街区去上班。如果他从不穿越（但可以碰到）从家到办公室的对角线，那么有多少条可能的道路?
类似题目：在圆上选择2n个点,将这些点成对连接起来使得所得到的n条线段不相交的方法数?



动态规划算法的基本要素

一、最优子结构

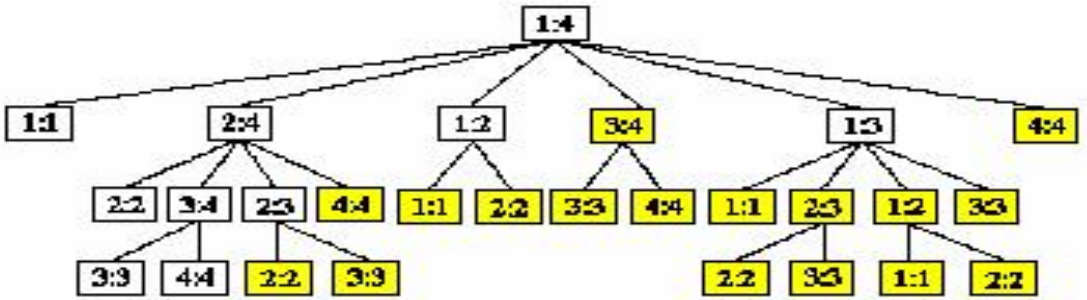
- 矩阵连乘计算次序问题的最优解**包含着其子问题的最优解**。这种性质称为**最优子结构性质**。
- 在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，**以自底向上的方式**递归地从子问题的最优解逐步构造出整个问题的最优解。最优子结构是问题能用动态规划算法求解的前提。

同一个问题可以有多种方式刻划它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）

动态规划算法的基本要素

二、重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质。**
- 动态规划算法，**对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。**
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。





动态规划算法的基本要素

三、备忘录方法

•备忘录方法的控制结构与直接递归方法的控制结构相同，区别在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看，**避免了相同子问题的重复求解**。

```
int LookupChain(int i,  int j){
      if (m[i][j] > 0) return m[i][j];
      if (i == j) return 0;
      int u = LookupChain(i,  i) + LookupChain(i+1,  j) + p[i-1]*p[i]*p[j];
      s[i][j] = i;
      for (int k = i+1; k < j; k++) {
        int t = LookupChain(i,  k) + LookupChain(k+1,  j) + p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k;}
        }
      m[i][j] = u;
      return u;   }
```



最长公共子序列

- 若给定序列$X=\{x_1,x_2,\ldots,x_m\}$，则另一序列$Z=\{z_1,z_2,\ldots,z_k\}$，
- Z是X的子序列是指存在一个严格递增下标序列$\{i_1,i_2,\ldots,i_k\}$使得对于所有$j=1,2,\ldots,k$有：$z_j=x_{i_j}$。
- 例如，序列Z={B，C，D，B}是序列X={A，B，C，B，D，A，B}的子序列，相应的递增下标序列为{2，3，5，7}。**(不需要连续，如何分析问题，分解子问题)**

- 给定2个序列X和Y，当另一序列Z既是X的子序列又是Y的子序列时，称Z是序列X和Y的**公共子序列**。

- 给定2个序列$X=\{x_1,x_2,\ldots,x_m\}$和$Y=\{y_1,y_2,\ldots,y_n\}$，找出X和Y的最长公共子序列。



最长公共子序列

给定2个序列X和Y，当另一序列Z既是X的子序列又是Y的子序列时，称Z是序列X和Y的**公共子序列**。

- 给定2个序列$X=\{x_1,x_2,\ldots,x_m\}$和$Y=\{y_1,y_2,\ldots,y_n\}$，找出X和Y的最长公共子序列。

- 解题：一开始肯定是比较；怎么比？
  解题的思路都是从简单开始的
  然后，有几种状态->子问题是什么



最长公共子序列

设序列$X=\{x_1,x_2,\ldots,x_m\}$和$Y=\{y_1,y_2,\ldots,y_n\}$的最长公共子序列为$Z=\{z_1,z_2,\ldots,z_k\}$，则：

(1)若$x_m=y_n$，则$z_k=x_m=y_n$，且z_{k-1}是x_{m-1}和y_{n-1}的最长公共子序列。

(2)若$x_m\neq y_n$且$z_k\neq x_m$，则Z是x_{m-1}和Y的最长公共子序列。

(3)若$x_m\neq y_n$且$z_k\neq y_n$，则Z是X和y_{n-1}的最长公共子序列。

只考虑当下与下一步

问题：如果能匹配，先不匹配；是不是后面有机会使得解更好

序列Z={B，C，D，B}是序列X={A，B，C，B，D，A，B}

由此可见，2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。因此，最长公共子序列问题具有**最优子结构性质。**

最长公共子序列：考虑S1方向的变化

```
for(i=1;i<=lena;i++){
    for(j=1;j<=lenb;j++)
        if(stra[i-1]==strb[j-1]) dp[i][j]=dp[i-1][j-1]+1;
        else dp[i][j]=MAX(dp[i-1][j],dp[i][j-1]);
}
```

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0									
2	3	0									
3	4	0									
4	5	0									
5	6	0									
6	7	0									
7	7	0									
8	8	0									

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	3	0	1								
3	4	0									
4	5	0									
5	6	0									
6	7	0									
7	7	0									
8	8	0									

相等，对角+1

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	3	0	1	1	1	1	1	1	1	1	1
3	4	0									
4	5	0									
5	6	0									
6	7	0									
7	7	0									
8	8	0									

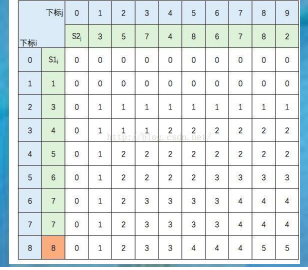
不等，二取大者

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	3	0	1	1	1	1	1	1	1	1	1
3	4	0	1	1	1	2	2	2	2	2	2
4	5	0									
5	6	0									
6	7	0									
7	7	0									
8	8	0									

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	3	0	1	1	1	1	1	1	1	1	1
3	4	0	1	1	1	2	2	2	2	2	2
4	5	0	1	2	2	2	2	2	2	2	2
5	6	0									
6	7	0									
7	7	0									
8	8	0									

下标i \ 下标j		0	1	2	3	4	5	6	7	8	9
	$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0
2	3	0	1	1	1	1	1	1	1	1	1
3	4	0	1	1	1	2	2	2	2	2	2
4	5	0	1	2	2	2	2	2	2	2	2
5	6	0	1	2	2	2	2	3	3	3	3
6	7	0									
7	7	0									
8	8	0									

最长公共子序列



下标i		下标j	0	1	2	3	4	5	6	7	8	9
		$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$		0	0	0	0	0	0	0	0	0	0
1	1		0	0	0	0	0	0	0	0	0	0
2	3		0	**1**	**1**	**1**	1	1	1	1	1	1
3	4		0	1	1	1	**2**	2	2	2	2	2
4	5		0	1	2	2	**2**	**2**	2	2	2	2
5	6		0	1	2	2	2	2	**3**	3	3	3
6	7		0	1	2	3	3	3	**3**	4	4	4
7	7		0	1	2	3	3	3	3	**4**	4	4
8	8		0	1	2	3	3	4	4	4	**5**	5

http://blog.csdn.net/

下标i		下标j	0	1	2	3	4	5	6	7	8	9
		$S2_j$	3	5	7	4	8	6	7	8	2	
0	$S1_i$		0	0	0	0	0	0	0	0	0	0
1	1		0	0	0	0	0	0	0	0	0	0
2	3		0	**1**	1	1	1	1	1	1	1	1
3	4		0	**1**	1	1	2	2	2	2	2	2
4	5		0	1	**2**	2	2	2	2	2	2	2
5	6		0	1	**2**	2	2	2	3	3	3	3
6	7		0	1	2	**3**	**3**	**3**	**3**	4	4	4
7	7		0	1	2	3	3	3	3	**4**	4	4
8	8		0	1	2	3	3	4	4	4	**5**	5

http://blog.csdn.net/

LCS = {3,4,6,7,8}；可以这样

只是2个例子，实际代码逻辑要有一致性

另一个方向的回推LCS ={3,5,7,7,8}

相同时逻辑：对角+1



子问题的递归结构

由最长公共子序列问题的最优子结构性质建立子问题最优值的递归关系。**用c[i][j]记录序列和的最长公共子序列的长度**。其中，$X_i=\{x_1,x_2,\ldots,x_i\}$；$Y_j=\{y_1,y_2,\ldots,y_j\}$。当i=0或j=0时，空序列是$X_i$和$Y_j$的最长公共子序列。故此时C[i][j]=0。其它情况下，由最优子结构性质可建立递归关系如下：

$$c[i][j]=\begin{cases}0 & i=0,j=0\\ c[i-1][j-1]+1 & i,j>0;x_i=y_j\\ \max\{c[i][j-1],c[i-1][j]\} & i,j>0;x_i\neq y_j\end{cases}$$



计算最优值

由于在所考虑的子问题空间中，总共有θ(mn)个不同的子问题，因此，用动态规划算法自底向上地计算最优值能提高算法的效率。

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b)
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++)  c[0][i] = 0;
    for (i = 1; i <= m; i++)
      for (j = 1; j <= n; j++) {
        if (x[i]==y[j]) {  c[i][j]=c[i-1][j-1]+1; b[i][j]=1;}
        else if (c[i-1][j]>=c[i][j-1]) { c[i][j]=c[i-1][j]; b[i][j]=2;}
        else { c[i][j]=c[i][j-1]; b[i][j]=3; }
      }
}
```

考虑，如何把自顶向下的递归转成自底向上的程序



计算最优值

构造最长公共子序列

```
void LCS(int i，int j，char *x，int **b)
{
    if (i ==0 || j==0) return;
    if (b[i][j]== 1){ LCS(i-1，j-1，x，b);
                      cout<<x[i]; }
    else if (b[i][j]== 2) LCS(i-1，j，x，b);
    else LCS(i，j-1，x，b);
}
```



算法的改进

•在算法lcsLength和lcs中，可进一步将数组b省去。事实上，数组元素c[i][j]的值仅由c[i-1][j-1]，c[i-1][j]和c[i][j-1]这3个数组元素的值所确定。对于给定的数组元素c[i][j]，可以不借助于数组b而仅借助于c本身在时间内确定c[i][j]的值是由c[i-1][j-1]，c[i-1][j]和c[i][j-1]中哪一个值所确定的。

•如果只需要计算最长公共子序列的长度，则算法的空间需求可大大减少。事实上，在计算c[i][j]时，只用到数组c的第i行和第i-1行。因此，用2行的数组空间就可以计算出最长公共子序列的长度。进一步的分析还可将空间需求减至O(min(m,n))。



最大子段和

•给定n个整数(可能为负整数)组成的序列$a_1,a_2,...a_n$，求该序列形如 $\sum_{k=i}^{j} a_k$ 的子段的最大值。

•例如，当(a1,a2 , a3 , a4 , a5 ,a6)=(-2,11,-4,13,-5,-2)时，
•最大子段和为：

•11+（-4）+13 =20



最大子段和

- 简单的方法：穷举**P59，穷举的改进：降低一维**

- 分治法：穷举**P60**-三种情况：
  - 1）左边的解就是解（分）
  - 2）右边的解就是解（分）
  - 3）左右都有一部分（合）

- 动态规划：b[j]=max{b[j-1]+a[j],a[j]},
  - 1<=j<=n
  - b[j-1]>0 or <0

- 例子：(-2,11,-4,13,-5,-2)

最大子段和

穷举法

```
int MaxSum（int n, int a*, int & besti, int & bestj）
{ int sum=0;
  for (int i=1; i<=n; i++)          //头部变化
     for (int j=i; j<=n; j++) {     //尾部变化
          int thissum=0;
                for (int k=i; k<=j; k++) thissum+=a[k];     //累计求和
                          if (thissum>sum) {
            sum= thissum;
                                besti= i;
                                bestj=j;
            }
        }
  return sum;
}
```

//这段求和；实际上每一次都比上次多一个元素；所以可以降维i->j；每次j往后推



最大子段和

从这个算法的3个for循环可以看出它所需的计算时间是$O(n^3)$事实上，如果我们注意到 $\sum_{k=i}^{j} a_k = a_j + \sum_{k=i}^{j-1} a_k$

则可将算法中的最后一个循环省去，避免重复计算，从而使算法得以改进。改进后的算法可描述为：

```
int MaxSum（int n, int *a, int & besti, int & bestj）
{ int sum=0;
 for (int i=1; i<=n; i++)
     {    int thissum=0;
          for (int j=i; j<=n; j++)
          { thissum += a[j];
            if (thissum>sum)
            { sum= thissum; besti= i; bestj=j; }
          }
      }
  return sum;
}
```

改进后的算法只需要$O(n^2)$的计算时间。

穷举法改进，降低一维

```
int MaxSum（int n, int a*, int & besti, int & bestj）
{ int sum=0;
  for (int i=1; i<=n; i++)          //头部变化
    for (int j=i; j<=n; j++) {      //尾部变化
        int thissum=0;
            for (int k=i; k<=j; k++) thissum+=a[k];     //累计求和
                if (thissum>sum) {
    sum= thissum;
                    besti= i;
                    bestj=j;
                }
        }
  return sum;
}
```



最大子段和

a[1:n]的最大子段和有三种情形：

(1) a[1:n]的最大子段和与a[1:n/2]的最大子段和相同。

(2) a[1:n]的最大子段和与a[n/2+1:n]的最大子段和相同。

(3) a[1:n]的最大子段和为 $\sum_{k=i}^{j} a_k$，且$1 \leqslant i \leqslant n/2$, $n/2+1 \leqslant j \leqslant n$。

分治法
第三种情况如何处理
从中间往两边求2个max



最大子段和

动态规划

```
for(i=1; i<count; i++)
{
   if(b[i-1]>0)  b[i]=b[i-1]+a[i];  //累计是正，有价值
   else  b[i]=a[i];    //前面累计负，无价值

   if(b[i]>max)  max=b[i];    //也可以计算完了for一次
}
```

b[j]=max{b[j-1]+a[j],a[j]},
1<=j<=n
b[j-1]>0 or <0

(-2,11,-4,13,-5,-2)

状态：前面的累计结果(目标是最大求和)，是否有利目标

-2,11,-4,13,-5,-2

一层for循环：O(n)

凸多边形最优三角剖分



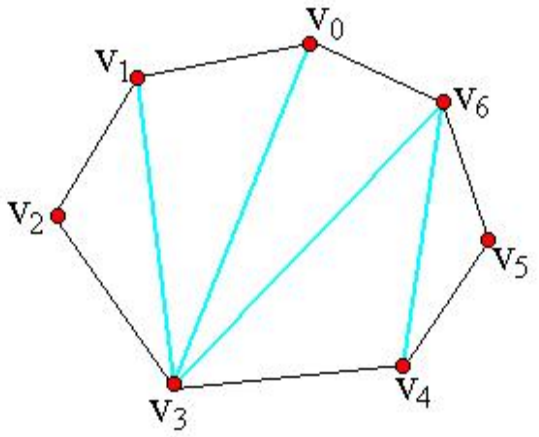
- 用多边形顶点的逆时针序列表示凸多边形，即$P=\{v_0,v_1,\ldots,v_{n-1}\}$表示具有n条边的凸多边形。
- 若v_i与v_j是多边形上不相邻的2个顶点，则线段v_iv_j称为多边形的一条弦。弦将多边形分割成2个多边形$\{v_i,v_{i+1},\ldots,v_j\}$和$\{v_j,v_{j+1},\ldots v_i\}$。
- 多边形的三角剖分是将多边形分割成互不相交的三角形的弦的集合T。
- 给定凸多边形P，以及定义在由多边形的边和弦组成的三角形上的权函数w。

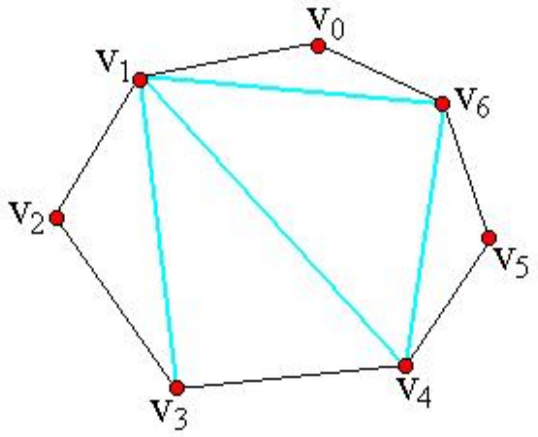
要求确定该凸多边形的三角剖分，使得该三角剖分中诸三角形上权之和为最小。

1、各点之间的点或弦的权值（如下图）

	1	2	3	4	5	6
1	0	2	2	3	1	4
2		0	1	5	2	3
3			0	2	1	4
4				0	6	2
5					0	1
6						0

2、通过三角形权函数w得到三角形的总权值






```
return weight[a][b] + weight[b][c] + weight[a][c];
```

三角剖分的结构及其相关问题



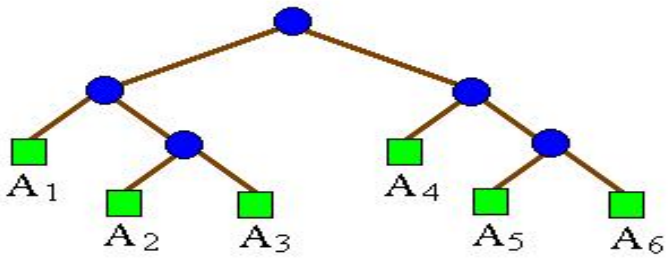
•一个表达式的**完全加括号方式相应于一棵完全二叉树**，称为表达式的语法树。例如，完全加括号的矩阵连乘积 $((A_1(A_2A_3))(A_4(A_5A_6)))$所相应的语法树如图 (a)所示。

•凸多边形$\{v_0,v_1,\ldots v_{n-1}\}$的三角剖分也可以用语法树表示。例如，**图(b)中凸多边形的三角剖分可用图 (a)所示的语法树表示。**

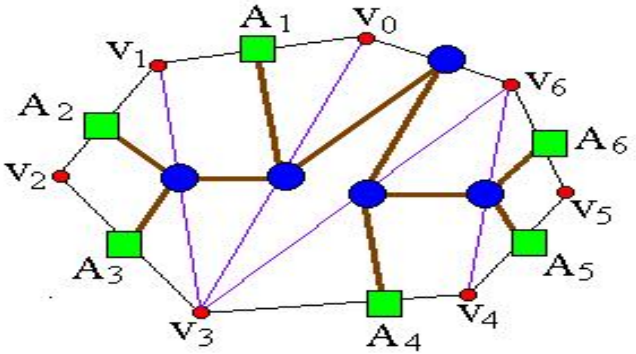
•矩阵连乘积中的每个矩阵**A_i对应于凸(n+1)边形中的一条边$v_{i-1}v_i$。**三角剖分中的一条弦v_iv_j，i<j，对应于矩阵连乘积A[i+1:j]。

两点组成边
加一点组成三角形

哪两条边结合
有个连续的关系
类似矩阵



(a)



(b)

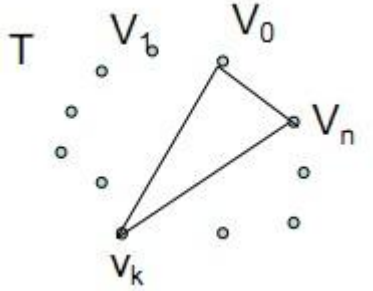
最优子结构性质



- 凸多边形的最优三角剖分问题有最优子结构性质。
- 事实上，若凸(n+1)边形$P=\{v_0,v_1,\ldots,v_{n-1}\}$的最优三角剖分T包含三角形$v_0v_kv_n$，$1\le k\le n-1$，则T的权为3个部分权的和：
  - 子多边形$\{v_0,v_1,\ldots,v_k\}$
  - 三角形v0vkvn的权，(多边形被三角形分割成两块)
  - 和$\{v_k,v_{k+1},\ldots,v_n\}$的权之和。

- 可以断言，由T所确定的这2个子多边形的三角剖分也是最优的。因为若有$\{v_0,v_1,\ldots,v_k\}$或$\{v_k,v_{k+1},\ldots,v_n\}$的更小权的三角剖分将导致T不是最优三角剖分的矛盾。

最优三角剖分的递归结构



•定义t[i][j]，1≤i<j≤n为凸子多边形{vi-1,vi,...,vj}的最优三角剖分所对应的权函数值，即其最优值。为方便起见，设退化的多边形{vi-1,vi}具有权值0。据此定义，要计算的凸(n+1)边形P的最优权值为t[1][n]。

•t[i][j]的值可以利用最优子结构性质递归地计算。当j-i≥1时，凸子多边形至少有3个顶点。由最优子结构性质，t[i][j]的值应为t[i][k]的值加上t[k+1][j]的值，再加上三角形$v_{i-1}v_kv_j$的权值，其中i≤k≤j-1。由于在计算时还不知道k的确切位置，而k的所有可能位置只有j-i个，因此可以在这j-i个位置中选出使t[i][j]值达到最小的位置。由此，t[i][j]可递归地定义为：

$$t[i][j]=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{t[i][k]+t[k+1][j]+w(v_{i-1}v_kv_j)\} & i<j\end{cases}$$

```
= t[i][k] + t[k+1][j] + Weight(i-1,k,j);
```

•**P67，算法同矩阵乘法**

```
return weight[a][b] + weight[b][c] + weight[a][c];
```

多边形游戏

多边形游戏是一个单人玩的游戏，开始时有一个由n个顶点构成的多边形。每个顶点被赋予一个整数值，每条边被赋予一个运算符 “+”或“*”。所有边依次用整数从1到n编号。

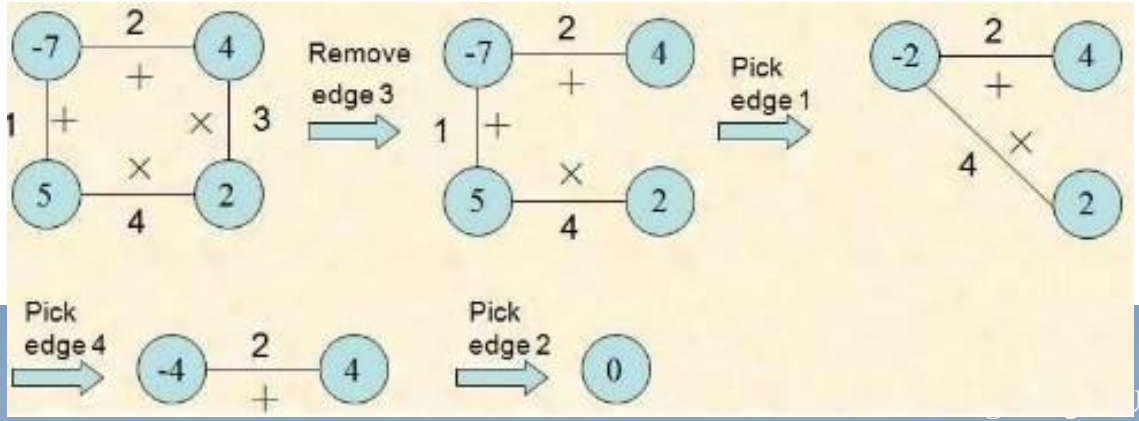
(0)游戏第1步，将一条边删除。随后n-1步按以下方式操作：

(1)选择一条边E以及由E连接着的2个顶点V1和V2；

(2)用一个新的顶点取代边E以及由E连接着的2个顶点V1和V2。将由顶点V1和V2的整数值通过边E上的运算得到的结果赋予新顶点。

(3)最后，所有边都被删除，游戏结束。游戏的得分就是所剩顶点上的整数值。

问题:对于给定的多边形，计算最高得分。







最优子结构性质

- 在所给多边形中，从顶点i(1≤i≤n)开始，长度为j(链中有j个顶点)的**顺时针链p(i，j)** 可表示为v[i]，op[i+1]，…，v[i+j-1]。
- 如果这条链的最后一次合并运算在op[i+s]处发生(1≤s≤j-1)，则可在op[i+s]处将链分割为2个子链p(i，s)和p(i+s，j-s)。（删除第一条不会分成2条链）
- 设m1是对子链p(i，s)的任意一种合并方式得到的值，而a和b分别是在所有可能的合并中得到的最小值和最大值。m2是p(i+s，j-s)的任意一种合并方式得到的值，而c和d分别是在所有可能的合并中得到的最小值和最大值。依此定义有a≤m1≤b，c≤m2≤d

(1)当op[i+s]='+'时，显然有a+c≤m≤b+d

(2)当op[i+s]='*'时，有min{ac，ad，bc，bd}≤m≤max{ac，ad，bc，bd}

- 换句话说，主链的最大值和最小值可由子链的最大值和最小值得到。
- **同时需要max、min，因为乘法运算可能*负负得正*。**
- **S在那个位置**

图像压缩（无损和有损）

图象的变位压缩存储格式将所给的象素点序列{p1,p2,...,pn},0≤pi≤255分割成m个连续段S1,S2,...,Sm。

第i个象素段Si中(1≤i≤m)，有l[i]个象素,且该段中每个象素最大只需用b[i]位表示。设 $t[i]=\sum_{k=1}^{i-1} l[k]$ 则第i个象素段Si为 $S_i=\{p_{t[i]+1},\cdots,p_{t[i]+l[i]}\}$




设 $h_i=\left\lceil \log\left(\max_{t[i]+1\le k\le t[i]+l[i]} p_k+1\right)\right\rceil$ ，则h_i≤b[i]≤8。

（1）因此需要用3位表示有效位数b[i]占用几位,

（2）如果限制1≤l[i]≤255，则需要用8位表示此段的连续长度l[i]/压缩的像素个数。



因此，第i个象素段所需的存储空间为l[i]*b[i]+11（=3+8）位。

按此格式存储象素序列{p1,p2,...,pn}，需要 $\sum_{i=1}^{m} l[i]*b[i]+11m$ 位的存储空间。

图象压缩问题要求确定象素序列{p1,p2,...,pn}的最优分段，使得依此分段所需的存储空间最少。每个分段的长度不超过256位。



图像压缩

设l[i]，b[i]，是{p1,p2,...,pn}的最优分段。显而易见，l[1]，b[1]是{p_1,...,$p_{l[1]}$}的最优分段，且l[i]，b[i]，是{$p_{l[1]+1}$,...,p_n}的最优分段。即图象压缩问题满足最优子结构性质。

设s[i]，1≤i≤n，是象素序列{p1,...,pi}的最优分段所需的存储位数。由最优子结构性质易知：

$$s[i]=\min_{1\le k\le \min\{i,256\}}\{s[i-k]+k*\mathrm{bmax}(i-k+1,i)\}+11$$

其中 $\mathrm{bmax}(i,j)=\left\lceil \log\left(\max_{i\le k\le j}\{p_k\}+1\right)\right\rceil$

算法复杂度分析：
由于算法**compress**中对k的循环次数不超这256，故对每一个确定的i，可在时间O(1)内完成的计算。因此整个算法所需的计算时间为O(n)。

s[i] 不同于前页的s
s[i-k]表示本次分割的前面部分解
bmax表示本段单个最大空间
K*bmax表示本段总和
11=3+8
K表示本次k个组合可能的结果

图像压缩

一个面包切成大小差不多8份；当然，每次只能切成2部分



1个子任务
+
1个单位切片



2个子任务



切钢条

一家公司购买长钢条，将其切割成短钢条出售，切割本身没有成本，长度为i的短钢条的价格为Pi。那给定一段长度为n的钢条和一个价格表Pi,求钢条的切割方案使得收益Rn最大。

长度i	1	2	3	4	5	6	7	8	9	10
价格P_i	1	5	8	9	10	17	17	20	24	30

```
for(i=1;i<=totalLen;i++)
    for(j=1;j<11&&i>=j;j++) {
        dp[i]=max2(dp[i],dp[i-j]+price[j]);
    }
```



1个子任务
+
1个单位切片



0-1背包问题

给定n种物品和一背包。物品i的重量是w_i，其价值为v_i，背包的容量为C。问应如何选择装入背包的物品，使得装入背包中物品的总价值最大?

0-1背包问题是一个特殊的整数规划问题。

$$\max \sum_{i=1}^{n} v_i x_i$$

$$\begin{cases} \sum_{i=1}^{n} w_i x_i \le C \\ x_i \in \{0,1\}, 1 \le i \le n \end{cases}$$

0-1背包问题-----递归

设所给0-1背包问题的子问题的最优值为m(i,j)：

$$\max \sum_{k=i}^{n} v_k x_k \qquad \begin{cases} \sum_{k=i}^{n} w_k x_k \le j \\ x_k \in \{0,1\}, i \le k \le n \end{cases}$$

即**m(i，j)**是背包容量为j，可选择物品为i，i+1，…，n时0-1背包问题的最优值。
由0-1背包问题的最优子结构性质，可以建立计算m(i，j)的递归式如下。

$$m(n,j) = \begin{cases} v_n & j \ge w_n \\ 0 & 0 \le j < w_n \end{cases}$$

这个是递归到最后了，剩余的j

$$m(i,j) = \begin{cases} \max\{m(i+1,j), m(i+1,j-w_i)+v_i\} & j \ge w_i \\ m(i+1,j) & 0 \le j < w_i \end{cases}$$

0-1背包问题----迭代

```
int nArr[6][13] = {{0}};     //递推数组
int nCost[6] = {0 , 2 , 5 , 3 , 10 , 4};  //花费或价值
int nVol[6]   = {0 , 1 ,  3 , 2 ,  6 , 2}; //物体体积
int bagV = 12;   //背包容量/
 for( int i = 1; i< sizeof(nCost)/sizeof(int); i++){
    for( int j = 1; j<=bagV; j++){     // j是剩余容量吗?
        if(j<nVol[i])
            nArr[i][j] = nArr[i-1][j];
        else
            nArr[i][j] = max(nArr[i-1][j] , nArr[i-1][j-nVol[i]] + nCost[i]);
    }                                  //不取第i个=i-1个的最优（上一行），
}                                      //取第i个+预留i所需空间后的前面行最大值
```

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2	2	2	2	2	2
2	0	2	2	5	7	7	7	7	7	7	7	7	7
3	0	2	3	5	7	8	10	10	10	10	10	10	10
4	0	2	3	5	7	8	10	12	13	15	17	18	20
5	0	2	4	6	7	9	11	12	14	16	17	19	21

0-1背包问题----迭代



行对应计算到某个物品；列对应到背包容量
对于1行：应为物品只有1个，所以不管空间多大，最优值都是2；同理对1列。

每个值都是对应(i,j)的最优值：
每个(i,j)计算的两个来源：max{【不取/上一行值】，【取/当前物品价值+剩余空间最优值/上一行剩余空间】}

对于蓝色7：

1、不取物品2：则=上一行同列的最优价值2
2、取物品2　：则=物品2的价值5+容量(当前容量4-物品2所需容量3)的最优值/即蓝色箭头位置

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	2	2	2	2	2	2	2	2	2	2	2	2
2	0	2	2	5	7	7	7	7	7	7	7	7	7
3	0	2	3	5	7	8	10	10	10	10	10	10	10
4	0	2	3	5	7	8	10	12	13	15	17	18	20
5	0	2	4	6	7	9	11	12	14	16	17	19	21

0-1背包问题----迭代

```
for( i=1; i<=m; i++)
  for( j=n; j>0; j-- ){    //倒序相同，因为是基于上一行计算的
       if ( j<b[i] )  a[i][j]=a[i-1][j];
      else {
            if(a[i-1][j]>(a[i-1][j-b[i]]+c[i])) a[i][j]=a[i-1][j];
            else a[i][j]=a[i-1][j-b[i]]+c[i];
      }
}
```

// 空间从n开始，过程是相同的

2	2	2	2	2	2	2	2	2	2	2	2
2	2	5	7	7	7	7	7	7	7	7	7
2	3	5	7	8	10	10	10	10	10	10	10
2	3	5	7	8	10	12	13	15	17	18	20
2	4	6	7	9	11	12	14	16	17	19	21

0-1背包问题----迭代

```
#include<iostream>
using namespace std;

int bagv=12,herb=6,tcost[15]= {0 , 1 , 3 , 2 ,   6 , 2},hvalue[15]= {0 , 2 , 5 , 3 , 10 , 4};
int f[200];
int i,j;

int main(){

    for(i=1;i<=herb;i++)
      for(j=bagv;j>=tcost[i];j--)
        f[j]=max(f[j],f[j-tcost[i]]+hvalue[i]);   //可以用一维数组；但必须逆序，因为后面会用到前面
    printf("%d\n",f[bagv]);
    return 0;
}
```

在二维里是上一行的前面，不影响；
但一维共用空间了

	0	1	2	3	4
0	0	0	0	0	0
1	0	2	2	2	2
2	0	2	2	5	7



0-1背包问题

算法复杂度分析：

从m(i，j)的递归式容易看出，算法需要O(nc)计算时间。当背包容量c很大时，算法需要的计算时间较多。例如，当$c>2^n$时，算法需要$\Omega(n2^n)$计算时间。

习题



在一个n行的点数值三角形中，寻找从顶点开始每一步可沿左斜（L）或右斜（R）向下至底的一条路径，使该路径所经过的点的数值和最小。

```
          22
        14  19
      30  25  10
    8  20  12  27
  6  25  32   6   4
6  10  10   6   2  32
32  29   2  13  15   3  24
```



习题

都说天上不会掉馅饼，但有一天gameboy正走在回家的小径上，忽然天上掉下大把大把的馅饼。说来gameboy的人品实在是太好了，这馅饼别处都不掉，就掉落在他身旁的10米范围内。馅饼如果掉在了地上当然就不能吃了，所以gameboy马上卸下身上的背包去接。但由于小径两侧都不能站人，所以他只能在小径上接。由于gameboy平时老呆在房间里玩游戏，虽然在游戏中是个身手敏捷的高手，但在现实中运动神经特别迟钝，每秒种只有在移动不超过一米的范围内接住坠落的馅饼。

为了使问题简化，假设在接下来的一段时间里，馅饼都掉落在0-10这11个位置。
开始时gameboy站在5这个位置，因此在第一秒，他只能接到4,5,6这三个位置中期中一个位置上的馅饼。
问gameboy最多可能接到多少个馅饼？（假设他的背包可以容纳无穷多个馅饼）